

Trabalho Prático 4

Grupo 22

Alexis Correia - A102495 João Fonseca - A102512

Problema 2

Enunciado

Este exercício é dirigido à prova de correção do algoritmo estendido de Euclides apresentado no trabalho TP3

1. Construa a asserção lógica que representa a pós-condição do algoritmo. Note que a definição da função $\gcd$ é $\gcd(a, b) \equiv \min \{ r > 0 \mid \exists s, t \quad r = a * s + b * t \}$.
2. Usando a metodologia do comando `{sf havoc}` para o ciclo, escreva o programa na linguagem dos comandos anotados (LPA). Codifique a pós-condição do algoritmo com um comando `assert`.
3. Construa codificações do programa LPA através de transformadores de predicados "strongest post-condition" e prove a correção do programa LPA.

Resolução

Primeiramente, vamos nos lembrar de como é o Algoritmo Estendido de Euclides:

```
{INPUT  a, b}
{assume  a > 0 and b > 0}

0: r, r', s, s', t, t' = a, b, 1, 0, 0, 1
1: while r' != 0:
2:   q = r div r'
3:   r, r', s, s', t, t' = r', r - q * r', s', s - q * s', t', t - q * t'

{OUTPUT r, s, t}
```

Parte 1

O output do Algoritmo Estendido de Euclides são os inteiros r , s e t . Pela definição do algoritmo e pela análise dos *outputs*, é fácil verificar que r é o Máximo Divisor Comum (gcd) dos *inputs* a e b e que $r = a \times s + b \times t$.

Por tanto, a asserção lógica que representa a pós-condição deste algoritmo é:

$$r = \gcd(a, b) \wedge r = a \times s + b \times t$$

Parte 2

Agora, para podermos escrever o Fluxo deste programa precisamos definir o invariante do ciclo. Com a análise cuidadosa do algoritmo, concluímos que o invariante do ciclo `while` é:

$$\text{gdc}(a,b)=\text{gdc}(r,r') \wedge r=a \times s+b \times t \wedge r'=a \times s'+b \times t'$$

Sendo assim, o Fluxo do programa (com o comando `havoc`) é:

```
[assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; ((assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume
False)||assume r' == 0 and inv); assert pos]
```

tal que:

- v é o vetor de todas as variáveis que ocorrem no corpo do ciclo;
- **pre** é a pré-condição: $a > 0 \wedge b > 0$;
- **pos** é a pós-condição: $r = \text{gcd}(a,b) \wedge r = a*s + b*t$;
- **inv** é o invariante definido anteriormente;

Parte 3

Em seguida utilizaremos os transformadores de predicados **Strongest Post-Condition** e depois provaremos a correção do programa.

```
[assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; ((assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume
False)||assume r' == 0 and inv); assert pos]
=
[assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; ((assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume
False)||assume r' == 0 and inv)] -> pos
=
[(assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume
False)||((assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v;
assume r' == 0 and inv))] -> pos
=
[assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume False]
or [assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v;
assume r' == 0 and inv] -> pos
=
([assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; assume
r' != 0 and inv; q=r div r';
```

```

r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv; assume False]
or [assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v;
assume r' == 0 and inv]) -> pos
=
([assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v; assume
r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt';assert inv;] and False)
or ([assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v;] and
r' == 0 and inv)) -> pos
=
([assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv; havoc v;
assume r' != 0 and inv; q=r div r';
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt'] -> inv) and False) or
(exists w.[assume pre; r,r',s,s',t,t'=a,b,1,0,0,1; assert inv][v/w]
and r' == 0 and inv)) -> pos
=*
((exists w'.(pre and r,r',s,s',t,t'=a,b,1,0,0,1 -> inv)[v/w'] and (b
and inv) and q=r div r' and
r,r',s,s',t,t'=r',r-qxr',s',s-qxs',t',t-qxt' -> inv) and False) or
(exists w.(pre and r,r',s,s',t,t'=a,b,1,0,0,1 -> inv)[v/w] and r' == 0
and inv)) -> pos
=
False or (exists w.(pre and r,r',s,s',t,t'=a,b,1,0,0,1 -> inv)[v/w]
and r' == 0 and inv) -> pos
=
exists w.(pre and r,r',s,s',t,t'=a,b,1,0,0,1 -> inv)[v/w] and r' == 0
and inv -> pos
=
(pre and r,r',s,s',t,t'=a,b,1,0,0,1 -> inv) and r' == 0 and inv -> pos

```

Por fim, podemos codificar as informações acima com o **pysmt**.

```

from pysmt.shortcuts import *
from pysmt.typing import *
#import math

def prove(f):
    with Solver(name="z3") as s:
        s.add_assertion(Not(f))
        if s.solve():
            print("Failed to prove.")
        else:
            print("Proved.")

a = Symbol('a',INT)
b = Symbol('b',INT)
r = Symbol('r',INT)
rl = Symbol('rl',INT)
s = Symbol('s',INT)

```

```

sl = Symbol('sl',INT)
t = Symbol('t',INT)
tl = Symbol('tl',INT)

pre = And(GT(a, Int(0)), GT(b, Int(0)))
inv = And(Equals(r, Times(a,s)+Times(b,t)), Equals(rl, Times(a,sl)
+Times(b,tl))) ## gcd
aux =
Implies(And(pre,Equals(r,a),Equals(rl,b),Equals(s,Int(1)),Equals(sl,In
t(0)),Equals(t,Int(0)),Equals(tl,Int(1))), inv)
pos = Equals(r, Times(a,s)+Times(b,t)) #And gcd

spc = Implies(And(aux, Equals(rl, Int(0)), inv), pos)
prove(spc)

Proved.

```